

Data Types & Type Casting

data types, garbage value, implicit and explicit conversion

When you write `int age = 20;`, that word `int` is doing two jobs at once. It tells the compiler what kind of value the box can hold, and it decides how much memory to reserve for it.

Data Types

DEFINITION

A data type tells the compiler **what kind of value** a variable can hold.

C++ groups them into three families.



For now everything you write will use the **primitive** family. The other two come later in the course.

TYPE	HOLDS	EXAMPLE
int	Whole numbers	int age = 20;
float	Decimal numbers	float pi = 3.14;
double	Decimal numbers, more precision	double g = 9.806;
char	A single character	char grade = 'A';
bool	true or false	bool pass = true;
void	No value at all	void print();

SIZES

Integers also come in different sizes — `short`, `long` and `long long`. The bigger the size, the wider the range of numbers it can store.

DATA TYPE	SIZE (BYTES)	SIZE (BITS)
char	1	8
bool	1	8
short	2	16
int	4	32
long	4	32
long long	8	64
float	4	32

DATA TYPE	SIZE (BYTES)	SIZE (BITS)
double	8	64
long double	16	—
void	—	—



`void` holds no value, so it has no size. A few sizes depend on the compiler and OS — for example `long` is 8 bytes on Linux and Mac. The values above are typical on Windows with MinGW.

You can always check on your own machine:

MAIN.CPP

```
cout << sizeof(int);    // 4
cout << sizeof(double); // 8
cout << sizeof(char);   // 1
```

Garbage Value

DEFINITION

A garbage value is the **random, unpredictable value** a variable holds when it is declared but not given any value.

Declaring a variable reserves a box in memory — but it does not clean the box out. Whatever bits were already sitting at that memory location stay there, and that leftover data is what you read.

MAIN.CPP

```
int x;           // declared, not initialized
cout << x;       // prints a garbage value
```

 The value is genuinely unpredictable. Run the same program twice and you may get two different numbers, so a bug caused by this can look like it disappears on its own. Always give your variables a starting value.

THE FIX

```
int x = 0;       // initialized – safe to use
cout << x;       // 0, every single time
```

Type Conversion

DEFINITION

Type conversion means **changing a value from one data type into another**.

It happens in two ways — either the compiler does it for you, or you ask for it directly.

Implicit

Done automatically by the compiler
— usually a smaller type into a
bigger one.

Explicit

Done manually by you. Also called
type casting.

IMPLICIT CONVERSION

The compiler widens the smaller type so no information is lost. Nothing needs to be written for this to happen.

MAIN.CPP

```
int a = 5;  
double b = a;      // implicit: int → double, b is 5.0
```

char → int → float → double
small —————→ large

widening is safe — nothing is lost

EXPLICIT CONVERSION

Here you are forcing a bigger type into a smaller one, so you have to ask for it. The compiler will not do it silently, because data **will** be lost.

MAIN.CPP

```
double x = 9.7;  
int y = (int) x;  // explicit: double → int, y is 9
```



Notice that `9.7` became `9`, not `10`. Converting a decimal to an integer **truncates** — it chops the decimal part off, it does not round.

	IMPLICIT	EXPLICIT
Who does it	The compiler, automatically	You, manually
Direction	Smaller into bigger	Bigger into smaller
Data loss	No	Yes — possible
Written as	Nothing to write	<code>(int)x</code>